JavaScript

From Beginner to Strong Practical and Interview Level

The complete 21-phase mentor guide: concepts, examples, practice tasks, mini quizzes, projects, interview preparation, final revision and answer key.

For learners who already know: basic HTML, CSS, Flexbox, Grid, responsive design and positioning.

How to Use This Guide

This guide follows one rule: **understand, then write the code yourself**. Reading is not learning; typing, breaking and fixing code is.

The loop for every concept

1. **Concept:** read what it is, why it exists, when to use it, how it works and common mistakes.
2. **Examples:** type every example into your own file. Change values and predict the result before running.
3. **Your Task:** attempt it alone first. Get stuck for a while; that is where learning happens.
4. **Review:** send your code to your mentor. Expect hints first, solutions only if needed.
5. **Mini Quiz:** answer on paper before checking the Answer Key at the end.
6. **Move forward** only when you can explain the concept in your own words and solve a new problem with it.

Setup: You only need a browser (Chrome or Edge) with DevTools, a code editor (VS Code) and a folder with index.html and app.js. Press F12 to open the Console.

Rule for the quizzes: The Answer Key is for checking after you commit to an answer. Looking first removes the learning.

Block	Phases	Focus
Foundations	0 - 3	Browser basics, variables, types, operators, conditions
Core logic	4 - 7	Loops, functions, scope, strings
Data	8 - 10	Arrays, objects, real data problems
The web page	11 - 13	DOM, events, forms
Async and network	14 - 15	Event loop, promises, async/await, fetch
Modern and robust	16 - 17	Modern syntax, errors, debugging
Apply and polish	18 - 20	15 projects, interview preparation, final revision

Contents

Phase 0: JavaScript & Browser Fundamentals 4
Phase 1: Variables, Values & Data Types 7
Phase 2: Operators & Expressions 10
Phase 3: Conditional Logic 12
Phase 4: Loops & Iteration 14
Phase 5: Functions 17
Phase 6: Scope, Hoisting & Execution Concepts 19
Phase 7: Strings 21
Phase 8: Arrays 23
Phase 9: Objects 27
Phase 10: Advanced Array + Object Problem Solving 30
Phase 11: The DOM 32
Phase 12: Events 35
Phase 13: Forms & Validation 38
Phase 14: Asynchronous JavaScript 40
Phase 15: Fetch & APIs 43
Phase 16: Modern JavaScript 45
Phase 17: Error Handling & Debugging 47
Phase 18: JavaScript Projects 49
Phase 19: Interview & Coding Preparation 52
Phase 20: Final Revision 54
Answer Key (Mini Quizzes) 58

Phase 0: JavaScript & Browser Fundamentals

0.1 What JavaScript is, and where it fits

You already know two languages of the web. **HTML** describes *what is on the page*. **CSS** describes *how it looks*. **JavaScript** describes *what the page does*: behavior, logic, reacting to the user. Think of a house: HTML is the walls and doors, CSS is the paint, JavaScript is the electricity and plumbing.

Why it exists

HTML and CSS cannot make decisions or remember things. They cannot check whether an email is valid, count cart items, or react to a click. JavaScript was created in 1995 to add logic and interactivity.

When we use it

- Reacting to clicks, typing, scrolling
- Validating forms before submission
- Updating the page without reloading
- Fetching data from servers (weather, products, messages)
- Building full applications (with Node.js it can even run servers)

How it works

JavaScript is a programming language. The computer cannot read it directly, so a **JavaScript engine** runs it. Chrome and Edge use **V8**, Firefox uses **SpiderMonkey**, Safari uses **JavaScriptCore**. When a page loads: (1) the browser downloads the HTML, (2) reads it top to bottom, (3) hands any JavaScript to the engine, (4) the engine executes it **line by line, top to bottom**.

Common mistakes and edge cases

- Java and JavaScript are **not** related. The name was marketing. They are as related as 'car' and 'carpet'.
- JavaScript is not browser-only (Node.js), but we learn it in the browser first.
- Users can turn JavaScript off, so critical content should still work in plain HTML.
- Your JavaScript runs on the user's machine and anyone can read it. Never put secrets in it.

YOUR TASK: Think and explore

1. In 2-3 sentences, explain HTML vs CSS vs JavaScript in your own words.
2. Name three apps you use and one thing JavaScript does in each.
3. Open Chrome, press F12, open the Console tab, type `2 + 2` and press Enter. Who read what you typed, and who calculated the answer?
4. Is Java the same as JavaScript? Why or why not?

0.2 The script tag: inline, internal and external JavaScript

The browser needs to be told where your JavaScript is. That is the job of the `<script>` tag. There are three ways to use it.

Style	What it looks like	When to use
Inline	JavaScript inside an HTML attribute, e.g. `onclick=...`	Almost never. It mixes behavior into markup and is hard to maintain.

Style	What it looks like	When to use
Internal	Code written between `<script>` and `</script>` in the HTML file	Quick experiments and tiny demos.
External	A separate `.js` file linked with `src`	Real projects. Cacheable, reusable, clean separation.

```
<!-- Internal -->
<script>
  console.log("Hello from internal JS");
</script>

<!-- External: file app.js sits next to index.html -->
<script src="app.js"></script>
```

Where to put the script tag

The browser reads HTML top to bottom. If a script that touches page elements runs *before* those elements exist, it fails. Two safe habits: put the script tag at the **end of the body**, or put it in the head with the `defer` attribute, which waits until the HTML is parsed.

```
<head>
  <script src="app.js" defer></script>
</head>
```

Common mistakes: Wrong file path in `src`; writing code inside a script tag that also has a `src` (the inline code is ignored); forgetting that the script runs before elements below it exist.

YOUR TASK: Your first page

1. Create `index.html` and `app.js` in one folder.
2. Link app.js with `defer`. Put `console.log('It works')` in app.js.
3. Open the page and confirm the message in the Console.
4. Move the script tag to the end of the body and remove defer. Does it still work? Why?

0.3 console.log(), comments and the developer console

`console.log()` prints values to the browser console. It is your main tool for seeing what your program is doing. The console also works as a live playground: you can type JavaScript there and it runs immediately.

```
console.log("Hello");          // a string
console.log(10 + 5);           // 15
console.log("Total:", 15);     // several values
console.warn("Careful");       // yellow warning
console.error("Broken");       // red error
console.table([{a:1},{a:2}]);  // table view
```

Comments

```
// single line comment
/* multi-line
   comment */
```

Comments are ignored by the engine. Use them to explain *why*, not to repeat *what* the code obviously does. Commenting out code is a quick way to disable it while debugging.

Basic debugging with the console

- Read the red error message: it names the error type, the message, the file and the line number.
- Use console.log to check the value of something at a specific moment.
- Common beginners' errors: misspelled names, missing quotes or brackets, wrong file path.

YOUR TASK: Console experiments

1. Print your name, then your age, then both in one console.log call.
2. Use console.error and console.table once each.
3. Deliberately write `console.lg('hi')`. Read the error. What does it tell you?

0.4 Strict mode and how JavaScript executes

Strict mode makes JavaScript less forgiving so mistakes become visible errors instead of silent bugs. Enable it by putting `'use strict';` at the top of a file or function. Modern code (modules, classes) is strict automatically.

```
'use strict';
x = 10;   // ReferenceError: x is not defined
          // (without strict mode this silently creates a global!)
```

How a script runs: the engine parses your code, then executes it from the first line to the last, one statement at a time. Each statement finishes before the next begins. This single-threaded, in-order model is the foundation for later topics (call stack, event loop).

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Name the three jobs: which of HTML, CSS, JavaScript decides what happens when a button is clicked?

Q2. Which is better for real projects: inline, internal or external JavaScript, and why?

Q3. What does the `defer` attribute do?

Q4. In what order does the engine run code?

Q5. Why is strict mode useful?

Phase 1: Variables, Values & Data Types

1.1 Variables: let, const and var

A **variable** is a named box in memory that holds a value. We use variables so the program can store, read and change data. **Declaration** creates the name. **Initialization** gives it the first value. **Reassignment** changes the value later.

```
let age;            // declaration (value is undefined)
age = 20;           // initialization
age = 21;           // reassignment

let city = "Delhi"; // declaration + initialization together
const PI = 3.14159; // const must be initialized immediately
// PI = 3;          // TypeError: Assignment to constant variable
```

Keyword	Reassign?	Redeclare?	Scope	Notes
let	Yes	No (same scope)	Block	Default choice for changing values
const	No	No	Block	Default choice for everything else
var	Yes	Yes	Function	Old style. Avoid in new code.

Why var is different and problematic

- **Function scope, not block scope:** a var declared inside an if or for block leaks out to the whole function.
- **Redeclaration allowed:** declaring the same name twice does not warn you, hiding typos and collisions.
- **Hoisting to undefined:** var is usable before its line and gives undefined instead of an error.
- **Global leakage:** top-level var attaches to the global object.

```
if (true) {
  var a = 1;
  let b = 2;
}
console.log(a); // 1  (leaked out of the block)
console.log(b); // ReferenceError (b lives only inside the block)
```

Important: const does not mean the value is frozen. It means the *binding* cannot be reassigned. A const object or array can still have its contents changed (we study this in Phase 9).

Naming conventions

- camelCase for variables and functions: `totalPrice`, `userName`
- UPPER_SNAKE_CASE for true constants: `MAX_USERS`
- Names are case-sensitive, cannot start with a digit, cannot be reserved words
- Use meaningful names: `price` beats `p`

YOUR TASK: Personal information program

1. Create variables for name, age, city, isStudent and favourite number.
2. Print each with a label using console.log.
3. Use const for things that never change and let for things that might.
4. Try reassigning a const and read the error.

1.2 Primitive data types and typeof

Every value in JavaScript has a **type**. The simplest ones are **primitives**: single, immutable values.

Type	Example	Notes
string	'hi', "hi", `hi`	Text
number	42, 3.14, -7, NaN, Infinity	One type for integers and decimals
boolean	true, false	Used in conditions
undefined	let x;	Variable declared but has no value
null	let y = null;	Intentional 'no value'
bigint	123n	Integers beyond safe number range
symbol	Symbol('id')	Unique identifier. Rarely needed now.

```
console.log(typeof "hello");   // "string"
console.log(typeof 42);        // "number"
console.log(typeof true);      // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);      // "object"  <-- a famous historical bug
console.log(typeof 10n);       // "bigint"
```

Edge cases worth knowing

- 0.1 + 0.2 is 0.30000000000000004 because numbers are binary floating point. Compare decimals carefully.
- NaN means 'not a number' but `typeof NaN` is 'number'. NaN is not equal to itself.
- undefined = JavaScript says there is no value yet. null = *you* say there is no value.
- Numbers beyond 2^53 - 1 lose precision; that is why bigint exists.

Primitive vs reference values (intro)

Primitives are copied by value: changing the copy does not affect the original. Objects and arrays are **reference** values: variables hold an address, so two variables can point to the same object. We return to this in depth in Phases 8 and 9.

```
let a = 5;
let b = a;   // copy of the value
b = 10;
console.log(a, b); // 5 10

let p = { n: 1 };
let q = p;   // copy of the reference
q.n = 99;
console.log(p.n);  // 99 (same object!)
```

YOUR TASK: Four mini programs

1. **Calculator:** store two numbers and print sum, difference, product, quotient and remainder.
2. **Temperature converter:** convert 37 C to Fahrenheit using F = C * 9/5 + 32.
3. **Bill calculator:** price per item, quantity, 18% tax; print subtotal, tax and total.
4. Use typeof to print the type of five different values you create.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `let a = 5; let b = a; b = 8; console.log(a);`

Q2. Why does `console.log(x); var x = 3;` print undefined but `console.log(y); let y = 3;` throws?

Q3. What does `typeof null` return and why is it surprising?

Q4. Difference between undefined and null?

Q5. Can you do `const arr = []; arr.push(1);`?

Phase 2: Operators & Expressions

2.1 Arithmetic, assignment and increment operators

An **expression** produces a value (2 + 3). An **operator** acts on values called operands.

Operator	Meaning	Example	Result
+ - * /	Basic math	7 / 2	3.5
%	Remainder (modulo)	7 % 2	1
**	Exponent	2 ** 3	8
= += -= *= /=	Assignment shortcuts	x += 5	x = x + 5
++ --	Increment / decrement	i++	i = i + 1

Prefix vs postfix: `++i` increments first and returns the new value; `i++` returns the old value then increments.

```
let i = 5;
console.log(i++); // 5  (then i becomes 6)
console.log(++i); // 7  (i becomes 7 first)
console.log(10 % 3);  // 1   useful for even/odd and wrap-around
```

2.2 Comparison, equality and type coercion

Comparison operators return booleans: `>` `<` `>=` `<=` `===` `!==`. The big topic is **== vs ===**.

- **===** (strict equality) compares value *and* type. No conversion.
- **==** (loose equality) converts types first, producing surprises.
- **Rule: use === and !== by default.**

```
console.log(5 === "5");   // false (number vs string)
console.log(5 == "5");    // true  (string converted to number)
console.log(0 == false);  // true  (surprising)
console.log(null == undefined);  // true
console.log(null === undefined); // false
console.log(NaN === NaN); // false (use Number.isNaN)
```

Type coercion

JavaScript automatically converts values between types. The + operator prefers strings if either side is a string; other math operators prefer numbers.

```
console.log("5" + 3);   // "53"  (string concatenation)
console.log("5" - 3);   // 2     (string converted to number)
console.log("5" * "2"); // 10
console.log(true + 1);  // 2
console.log(Number("42"), Number("abc")); // 42 NaN
```

2.3 Logical operators, truthy/falsy, ?? and the ternary

Falsy values (convert to false): `false, 0, -0, 0n, '', null, undefined, NaN`. **Everything else is truthy**, including `'0'`, `'false'`, `[]` and `{}`.

Operator	Behavior	Example
&& (AND)	Returns first falsy value, otherwise the last value	'a' && 'b' gives 'b'
|| (OR)	Returns first truthy value, otherwise the last	0 || 'x' gives 'x'
! (NOT)	Flips truthiness to a boolean	!'' gives true
?? (nullish)	Right side only if left is null or undefined	0 ?? 5 gives 0

```
let qty = 0;
console.log(qty || 10);  // 10  (0 is falsy, probably not what you want)
console.log(qty ?? 10);  // 0   (only null/undefined trigger the default)

let age = 20;
let status = age >= 18 ? "Adult" : "Minor";   // ternary
```

Operator precedence

Order of evaluation, high to low (simplified): parentheses, `++ --` `!`, `**`, `* / %`, `+ -`, comparisons, ===, &&, ||, ??, ternary, assignment. When in doubt, add parentheses.

Template literals

```
const name = "Asha", n = 3;
console.log(`Hi ${name}, you have ${n * 2} messages.`); // expressions allowed
console.log(`line 1
line 2`);   // multi-line strings work
```

YOUR TASK: Four small programs

1. **Grade calculator:** given marks in 5 subjects, print total, percentage and a grade using nested ternaries or &&.
2. **Eligibility checker:** a person can vote if age >= 18 and is a citizen; print a message with template literals.
3. **Login simulation:** username and password stored in variables; print whether the login succeeds.
4. **Discount calculator:** 10% off above 1000, 20% off above 5000; print the final amount.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Predict: `console.log('5' + 2, '5' - 2)`

Q2. Predict: `console.log(0 || 'a', 0 ?? 'a')`

Q3. Predict: `console.log([] == false, [] ? 't' : 'f')`

Q4. Predict: `let i = 2; console.log(i++ + ++i)`

Q5. Why is NaN === NaN false and how do you test for NaN?

Phase 3: Conditional Logic

3.1 if, else if, else

Programs need to make decisions. A **condition** is any expression; JavaScript converts it to true or false (truthy/falsy) and picks a path. Only the **first** matching branch in an if / else if chain runs.

```
const marks = 72;
if (marks >= 90) {
  console.log("A");
} else if (marks >= 75) {
  console.log("B");
} else if (marks >= 60) {
  console.log("C");     // this runs
} else {
  console.log("Fail");
}
```

Common mistakes

- Using = instead of === inside a condition (assignment, not comparison).
- Wrong order: checking `marks >= 60` before `marks >= 90` makes the higher branches unreachable.
- Forgetting braces and adding a second line that is not actually inside the if.
- Comparing strings of numbers without converting: `'10' > '9'` is false (string comparison).

3.2 Combining conditions, nesting and guard clauses

```
// combining
if (age >= 18 && hasId) { /* allow */ }
if (day === "Sat" || day === "Sun") { /* weekend */ }

// deeply nested (hard to read)
if (user) {
  if (user.active) {
    if (user.balance > 0) { pay(); }
  }
}

// guard clauses (flat and clear)
function pay(user) {
  if (!user) return;
  if (!user.active) return;
  if (user.balance <= 0) return;
  // main logic here
}
```

Guard clauses handle invalid cases early and return, so the main logic stays unindented and easy to follow.

3.3 switch and the ternary

```
const day = 3;
switch (day) {
  case 1:
    console.log("Mon");
    break;           // without break, execution FALLS THROUGH to next case
  case 2:
  case 3:            // intentional grouping
    console.log("Tue or Wed");
    break;
  default:
    console.log("Other");
}
```

Use **switch** when comparing one value against many exact options (it uses strict equality). Use **if / else if** for ranges and complex conditions. Use the **ternary** only for short, simple choices; avoid nesting them.

YOUR TASK: Decision programs

1. **Grade system:** marks to grade using if / else if.
2. **ATM logic:** check PIN, check balance, check withdrawal is a multiple of 100, then withdraw. Use guard clauses.
3. **Age category:** child, teen, adult, senior.
4. **Electricity bill:** first 100 units at 3, next 100 at 5, above 200 at 8 per unit. Print the bill.
5. **Ticket pricing:** price depends on age and whether it is a weekend. Use switch for the day type.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Predict: `if ('0') console.log('A'); else console.log('B');`

Q2. What happens in a switch if you forget break?

Q3. Why can this fail? `if (x = 5) {}`

Q4. Rewrite nested ifs into guard clauses. What is the benefit?

Q5. When would you choose switch over else if?

Phase 4: Loops & Iteration

4.1 for, while and do...while

A **loop** repeats code. Every loop has three ideas: a **starting state**, a **condition** that decides whether to continue, and an **update** that moves toward ending.

```
// for: when you know how many times
for (let i = 1; i <= 5; i++) {
  console.log(i);
}

// while: when you repeat until something changes
let n = 100;
while (n > 1) {
  n = n / 2;
}

// do...while: body runs at least once
let tries = 0;
do {
  tries++;
} while (tries < 3);
```

Choosing the right loop

Situation	Loop
Known count or index range	for
Unknown count, depends on a condition	while
Must run once before checking (e.g. menu, retry)	do...while
Every item of an array or string	for...of (see 4.3)

Loop tracing (the skill that makes you good)

Tracing means writing out the variables at each iteration, like a table. Do this on paper for any loop you do not fully understand.

```
let sum = 0;
for (let i = 1; i <= 3; i++) { sum += i; }

// i | sum before | sum after
// 1 |     0      |    1
// 2 |     1      |    3
// 3 |     3      |    6
```

Common mistakes

- **Infinite loops:** forgetting to update the variable, or an update in the wrong direction (`while (i > 0) { i++ }`).
- **Off-by-one errors:** < versus <= and starting at 0 versus 1.
- Changing the loop variable inside the body unexpectedly.
- Declaring accumulator variables inside the loop so they reset each time.

If your browser tab freezes: an infinite loop is the likely culprit. Close the tab and re-check the condition and update.

4.2 break, continue and nested loops

```
for (let i = 1; i <= 10; i++) {
  if (i === 3) continue;  // skip this iteration only
  if (i === 6) break;     // exit the whole loop
  console.log(i);         // 1 2 4 5
}

// nested: the inner loop completes for each outer iteration
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= row; col++) line += "*";
  console.log(line);      // *  **  ***
}
```

4.3 for...of and for...in

```
const fruits = ["apple", "banana", "mango"];
for (const f of fruits) console.log(f);        // values

const user = { name: "Asha", age: 20 };
for (const key in user) console.log(key, user[key]);   // keys

for (const ch of "hey") console.log(ch);       // h e y
```

	for...of	for...in
Gives you	Values	Keys (property names)
Use on	Arrays, strings, Sets, Maps	Objects
Avoid on	Plain objects	Arrays (order and inherited keys are unreliable)

YOUR TASK: Solve manually first (no shortcuts yet)

1. **Number patterns:** print a right triangle of stars, then a pyramid.
2. **Multiplication table** of any number up to 10.
3. **Sum and count:** sum of 1 to 100; count how many numbers 1-100 are divisible by 7.
4. **Searching:** given an array, find if a value exists using a loop and break.
5. **Filtering:** from an array of numbers, build a new array of only the even ones.
6. **Frequency counting:** count how many times each character appears in a string using an object.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `for (let i = 0; i < 3; i++) console.log(i * 2);`

Q2. Which loop always runs at least once?

Q3. Difference between break and continue?

Q4. How many times does the inner line run? `for(i=0;i<3;i++) for(j=0;j<4;j++) x++;`

Q5. When do you use for...in and for...of?

Phase 5: Functions

5.1 What functions are and how to write them

A **function** is a named, reusable block of code that takes input (parameters), does work, and may give back output (return value). Functions let you avoid repetition (DRY), break problems into small parts, name ideas, and test logic in isolation.

```
// declaration
function add(a, b) {        // a, b are PARAMETERS
  return a + b;
}
console.log(add(2, 3));     // 2, 3 are ARGUMENTS -> 5

// expression
const multiply = function (a, b) { return a * b; };

// arrow function
const square = (n) => n * n;
const greet = name => `Hi ${name}`;

// default parameter
function power(base, exp = 2) { return base ** exp; }
```

Return vs print

`return` hands a value back to the caller and **immediately ends** the function. `console.log` just displays something. A function without return gives back `undefined`. Prefer returning values and printing outside, so your function stays reusable.

Common mistakes

- Writing code after return (it never runs).
- Calling `add` without parentheses (you get the function itself, not its result).
- Forgetting return, then getting undefined.
- In arrow functions with braces, forgetting that you must write return explicitly.

5.2 Scope inside functions, pure vs impure

Variables declared inside a function are **local**: invisible outside it. Functions can read variables from outside (global or enclosing). A function is **pure** if it depends only on its inputs and has no side effects: same input always gives same output. **Impure** functions change outside state, print, read time/randomness, or modify their arguments. Prefer pure functions for logic; keep impure ones at the edges.

```
let total = 0;
function addToTotal(n) { total += n; }   // impure: changes outside state
function sum(a, b) { return a + b; }      // pure
```

5.3 Callbacks, higher-order functions, composition

In JavaScript functions are **values**: you can store them in variables, pass them as arguments and return them. A function passed into another function is a **callback**. A function that takes or returns a function is a **higher-order function**.

```
function repeat(times, action) {      // higher-order
  for (let i = 0; i < times; i++) action(i);
}
repeat(3, (i) => console.log("Run", i));   // arrow function as callback

function makeMultiplier(m) {           // returns a function
  return (n) => n * m;
}
const double = makeMultiplier(2);
console.log(double(5));                // 10

// composition: feed one result into the next
const compose = (f, g) => (x) => f(g(x));
const addOne = x => x + 1;
console.log(compose(double, addOne)(4)); // double(addOne(4)) = 10
```

Rest parameters and spread

```
function total(...nums) {             // rest: collects arguments into an array
  let s = 0;
  for (const n of nums) s += n;
  return s;
}
console.log(total(1, 2, 3, 4));       // 10

const arr = [3, 1, 2];
console.log(Math.max(...arr));        // spread: expands array into arguments
```

YOUR TASK: Build a toolbox of functions

1. **Calculator functions:** add, subtract, multiply, divide (guard against divide by zero).
2. **Validation functions:** isEven(n), isAdult(age), isValidPassword(str) (min 8 chars).
3. **Utility functions:** getMax(...nums), average(...nums), capitalize(word).
4. **Callback practice:** write applyTwice(fn, value) so applyTwice(x => x + 3, 10) returns 16.
5. **Mini project:** a tip and bill splitter. Functions: calcTip, calcTotal, splitBill. Compose them for the final result.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `function f(){ return; } console.log(f());`

Q2. Difference between parameter and argument?

Q3. Is this pure? `const f = (x) => x + Math.random();`

Q4. What is a callback?

Q5. What does `(a) => a * 2` return when called with 4?

Phase 6: Scope, Hoisting & Execution Concepts

6.1 Scope: where variables live

Scope is the region of code where a name is visible. JavaScript has global scope, function scope and block scope, and uses **lexical scope**: visibility is decided by where code is *written*, not where it is called.

Scope	Created by	let / const	var
Global	Top of a script	Visible everywhere	Visible everywhere (and on window)
Function	function body	Local to function	Local to function
Block	{ } of if, for, while, plain blocks	Local to block	Ignores blocks (leaks out)

Scope chain

When you use a name, JavaScript looks in the current scope; if absent, in the enclosing scope, and so on up to global. This lookup path is the **scope chain**. If it is never found: ReferenceError.

```
const a = "global";
function outer() {
  const b = "outer";
  function inner() {
    const c = "inner";
    console.log(a, b, c);   // finds a, b by walking up the chain
  }
  inner();
}
outer();
// console.log(b);  // ReferenceError: b is not defined here
```

Shadowing: an inner variable with the same name hides the outer one. It is legal but confusing; avoid it.

6.2 Hoisting and the temporal dead zone

Before running, the engine scans each scope and registers declarations. This is why you can sometimes use things before the line that declares them.

Declaration	Before its line
function declaration	Fully usable (name and body hoisted)
var	Exists, value is undefined
let / const	Exists but unusable: ReferenceError (temporal dead zone)
function expression / arrow in var/let/const	Follows the variable's rule (not callable early)

```
sayHi();                      // works: "Hi"
function sayHi() { console.log("Hi"); }

console.log(x);               // undefined (var hoisted)
var x = 5;

// console.log(y);            // ReferenceError (TDZ)
let y = 5;

// hello();                   // TypeError: hello is not a function
var hello = function () {};   // only the variable name was hoisted
```

Takeaway: Hoisting is not 'code moving to the top'. It is the engine registering declarations first. Declare variables before use and the confusion disappears.

6.3 Execution context, call stack, memory model

When code runs, the engine creates an **execution context**: a record holding the variables, scope chain and the value of this for that piece of code. There is one **global** context, and one new **function** context for every function call.

The **call stack** tracks which context is running. Calling a function pushes a context; finishing it (return) pops it. The top of the stack is what runs now. JavaScript has one call stack, so it does one thing at a time.

```
function a() { b(); console.log("a done"); }
function b() { c(); console.log("b done"); }
function c() { console.log("c done"); }
a();
// stack: global -> a -> b -> c   (c finishes first)
// output: c done, b done, a done
```

Basic memory model: primitives are stored directly in the variable. Objects and arrays live in the **heap**; the variable stores a reference (address) to them. A **stack overflow** happens when functions call themselves without end (infinite recursion).

YOUR TASK: Predict, then verify in the browser

1. Write five snippets mixing var, let, const, blocks and functions. Predict each output before running.
2. Draw the call stack on paper for three nested function calls.
3. Explain in your own words why function declarations work before their line but let does not.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `{ var a = 1; let b = 2; } console.log(typeof a, typeof b);`

Q2. Output? `console.log(f()); function f(){ return 'ok'; }`

Q3. Output? `var v = 1; function t(){ console.log(v); var v = 2; } t();`

Q4. What is lexical scope?

Q5. What causes a stack overflow?

Phase 7: Strings

7.1 Indexing, length and immutability

A string is an ordered sequence of characters. Positions are counted from **0**. Strings are **immutable**: methods never change the original; they return a new string.

```
const s = "JavaScript";
console.log(s.length);     // 10
console.log(s[0]);         // "J"
console.log(s[s.length-1]);// "t"  (last character)
console.log(s.at(-1));     // "t"
s[0] = "j";                // silently ignored (immutable)
console.log(s);            // "JavaScript"
```

7.2 Common string methods

Method	What it does	Example result
toUpperCase / toLowerCase	Change case	'Hi'.toUpperCase() gives 'HI'
trim	Removes spaces at both ends	' a '.trim() gives 'a'
includes(x)	true if x is inside	'hello'.includes('ell') gives true
startsWith / endsWith	Check beginning / end	'a.js'.endsWith('.js') gives true
indexOf(x)	Position of first match, or -1	'hello'.indexOf('l') gives 2
slice(a, b)	Extract from a up to (not including) b; negatives allowed	'hello'.slice(1,3) gives 'el'
substring(a, b)	Like slice but swaps args if a > b and treats negatives as 0	'hello'.substring(1,3) gives 'el'
replace(x, y)	Replaces first match	'a-b-c'.replace('-', '+') gives 'a+b-c'
replaceAll(x, y)	Replaces all matches	'a-b-c'.replaceAll('-', '+') gives 'a+b+c'
split(sep)	Cuts into an array	'a,b'.split(',') gives ['a','b']
concat / +	Join strings (template literals are usually nicer)	'a'.concat('b') gives 'ab'

Because methods return new strings, you can **chain** them: `name.trim().toLowerCase()`. Remember to store the result: `s.toUpperCase();` on its own changes nothing.

Common mistakes and edge cases

- Expecting the original string to change.
- Case-sensitive comparisons: normalize with toLowerCase() before comparing.
- indexOf returns -1 (truthy!) when not found; compare explicitly with `!== -1` or use includes.
- Empty string is falsy; a string of spaces is truthy. Trim before validating.

YOUR TASK: String problems

1. **String validator:** check that an input is not empty after trimming and has no spaces inside.
2. **Username validator:** 4-12 characters, starts with a letter, only letters/digits/underscore (use loops and char checks first).
3. **Word counter:** count words in a sentence, ignoring extra spaces.
4. **Palindrome:** check whether a word reads the same backwards, ignoring case.
5. **Character frequency:** print how many times each character appears.
6. **Text formatter:** turn 'hELLO wORLD' into 'Hello World'.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `'hello'.slice(-3)`

Q2. Output? `let s = 'abc'; s.toUpperCase(); console.log(s);`

Q3. Output? `'a b c'.split(' ').length`

Q4. Why is `if ('abc'.indexOf('a'))` a bug?

Q5. Replace all dashes in 'a-b-c' without regex?

Phase 8: Arrays

8.1 Creating, reading and changing arrays

An **array** is an ordered list of values, indexed from 0. It can hold any types. It is a **reference** value: variables hold an address to the list in memory.

```
const nums = [10, 20, 30];
console.log(nums[0]);        // 10
console.log(nums.length);    // 3
console.log(nums[nums.length - 1]);  // 30 (last)
nums[1] = 99;                // update by index -> [10, 99, 30]
console.log(nums[10]);       // undefined (no such index)
```

8.2 Adding, removing and copying

Method	What it does	Mutates?	Returns
push(x)	Add to end	Yes	New length
pop()	Remove from end	Yes	Removed item
unshift(x)	Add to start	Yes	New length
shift()	Remove from start	Yes	Removed item
splice(i, del, ...add)	Remove/insert anywhere	Yes	Array of removed items
slice(a, b)	Copy a portion (b excluded)	No	New array
concat(...)	Join arrays	No	New array
reverse()	Reverse order	Yes	Same array
sort(fn)	Sort	Yes	Same array

slice vs splice: slice **copies** and leaves the original alone (think 'slice a piece off a picture'). splice **edits** the original (think 'splice a film reel'). Mixing them up is the most common array bug.

```
const a = [1, 2, 3, 4, 5];
console.log(a.slice(1, 3)); // [2, 3]      a is still [1,2,3,4,5]
console.log(a.splice(1, 2));// [2, 3]      a is now [1,4,5]
a.splice(1, 0, "x");        // insert: a is [1,"x",4,5]
a.push(6);                  // a is [1,"x",4,5,6]
```

8.3 Searching: includes, indexOf, find, findIndex, some, every

Method	Returns	If nothing matches
includes(x)	true / false	false
indexOf(x)	Index of first match	-1
find(fn)	First **item** where fn is true	undefined
findIndex(fn)	First **index** where fn is true	-1
some(fn)	true if **at least one** item passes	false (empty array: false)
every(fn)	true if **all** items pass	true for an empty array (vacuous truth)

```
const users = [{id:1, age:15}, {id:2, age:22}, {id:3, age:30}];
console.log(users.find(u => u.age >= 18));       // {id:2, age:22}
console.log(users.findIndex(u => u.age >= 18));  // 1
console.log(users.some(u => u.age >= 18));       // true
console.log(users.every(u => u.age >= 18));      // false
```

8.4 Transforming: forEach, map, filter, reduce

Method	Purpose	Returns	Mutates original?
forEach(fn)	Do something for each item (side effects)	undefined	No (unless you change items yourself)
map(fn)	Transform each item into a new item	New array, same length	No
filter(fn)	Keep items where fn is true	New array, shorter or equal	No
reduce(fn, init)	Combine all items into one value	Any single value	No

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);          // [2, 4, 6, 8]
const evens = nums.filter(n => n % 2 === 0);   // [2, 4]
const total = nums.reduce((sum, n) => sum + n, 0);  // 10
nums.forEach(n => console.log(n));             // prints, returns undefined
console.log(nums);   // [1, 2, 3, 4] unchanged
```

How reduce works

The callback receives **accumulator** (result so far) and **current item**. Whatever you return becomes the accumulator for the next item. The second argument is the starting accumulator.

```
// step by step for [1,2,3,4] with start 0:
// acc=0, n=1 -> 1
// acc=1, n=2 -> 3
// acc=3, n=3 -> 6
// acc=6, n=4 -> 10   final result
```

The differences you must be able to explain

Pair	Difference
map vs forEach	map RETURNS a new array of results; forEach returns nothing and is for side effects.
filter vs find	filter returns an array of ALL matches (maybe empty); find returns the FIRST match or undefined.
find vs findIndex	find gives the item; findIndex gives its position (or -1).
slice vs splice	slice copies without mutation; splice mutates the original.
some vs every	some: at least one passes. every: all must pass.
reduce vs a loop	reduce is a loop that folds items into one value in an expression. A for loop is more flexible; use reduce when the result is a single accumulated value and stay with loops when logic gets complicated.

Sort, flat, join

```
console.log([10, 9, 1].sort());          // [1, 10, 9]  default = as STRINGS!
console.log([10, 9, 1].sort((a, b) => a - b)); // [1, 9, 10]  numeric ascending
console.log([[1, 2], [3]].flat());       // [1, 2, 3]
console.log(["a", "b"].join("-"));       // "a-b"
```

Time intuition: indexOf, includes, find, filter, map, some, every, reduce all walk the list: O(n). push and pop are O(1); shift and unshift are O(n) because every item moves. sort is about O(n log n). Chaining three methods is 3 passes; fine for normal data.

Common mistakes

- Using map when you do not need the new array (use forEach).
- Forgetting to return inside a braces-bodied arrow function in map/filter.
- Default sort() on numbers.
- Mutating with sort/reverse/splice when you needed the original too (copy first with `[...arr]`).
- Assuming `const b = a` copies an array. It does not; it copies the reference.

YOUR TASK: Array practice

1. **Student marks analysis:** given an array of marks, print highest, lowest, average, how many passed (>= 40).
2. **Product filtering:** array of products (name, price, inStock). Filter in-stock items under a price.
3. **Shopping cart:** add, remove (by name) and total with reduce.
4. **Expense tracker:** list of expenses; total, biggest, and a list of only the names.
5. **Frequency counting:** count word occurrences in an array using reduce.
6. **Data transformation:** turn [' Asha ', 'RAVI'] into ['asha', 'ravi'] using map.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `const a=[1,2,3]; const b=a.map(n=>n*2); console.log(a, b);`

Q2. Output? `[1,2,3].forEach(n => n * 2)` - what does the expression return?

Q3. Output? `const a=[1,2,3,4,5]; a.splice(1,2); console.log(a);`

Q4. Output? `[].every(x => x > 5)`

Q5. Output? `const a=[1,2]; const b=a; b.push(3); console.log(a);`

Q6. How would you sort numbers [5,1,10] ascending correctly?

Phase 9: Objects

9.1 Creating objects and accessing properties

An **object** stores related data as key-value pairs. Use arrays for ordered lists of similar things; use objects for one thing with named parts. A function stored as a property is a **method**.

```
const student = {
  name: "Asha",
  age: 20,
  "favourite color": "blue",     // quotes needed for odd keys
  greet() { return `Hi, ${this.name}`; }   // method
};

console.log(student.name);                 // dot notation
console.log(student["favourite color"]);   // bracket notation
const key = "age";
console.log(student[key]);                 // bracket allows variables

student.city = "Delhi";     // add
student.age = 21;           // update
delete student.city;        // delete
console.log("age" in student);             // true
```

Dot vs bracket: use dot when you know the key at write time. Use brackets when the key is in a variable, computed, or not a valid identifier.

9.2 Nesting, destructuring, shorthand, optional chaining

```
const order = {
  id: 7,
  customer: { name: "Ravi", address: { city: "Pune" } },
  items: [{ sku: "A1", qty: 2 }, { sku: "B2", qty: 1 }]
};
console.log(order.customer.address.city);   // "Pune"
console.log(order.items[0].qty);            // 2

// optional chaining: stops safely if something is missing
console.log(order.customer.phone?.number);  // undefined (no error)
console.log(order.coupon?.code ?? "none");  // "none"

// destructuring
const { id, customer: { name } } = order;
const [first, second] = [10, 20];
const { qty = 1 } = order.items[0];         // default value

// shorthand and computed keys
const age = 20;
const p = { name, age };                    // same as { name: name, age: age }
const field = "score";
const s = { [field]: 90 };                  // { score: 90 }
```

9.3 Object helpers and copying

Tool	Returns
Object.keys(o)	Array of keys
Object.values(o)	Array of values
Object.entries(o)	Array of [key, value] pairs
Object.assign(target, src)	Copies src properties into target (mutates target)
{ ...o, extra: 1 }	New object (shallow copy) with changes

```
const price = { pen: 10, book: 50 };
for (const [item, cost] of Object.entries(price)) console.log(item, cost);
const total = Object.values(price).reduce((a, b) => a + b, 0);   // 60
```

Reference behavior (very important)

```
const a = { n: 1 };
const b = a;           // same object, two names
b.n = 2;
console.log(a.n);      // 2

const c = { ...a };    // new object (shallow copy)
c.n = 3;
console.log(a.n);      // 2

console.log({} === {});    // false: different objects
console.log(a === b);      // true: same reference

const deep = { user: { name: "A" } };
const shallow = { ...deep };
shallow.user.name = "Z";
console.log(deep.user.name);  // "Z"! nested objects are still shared
// real deep copy: structuredClone(deep)
```

Also remember: const object = { } stops you reassigning the variable but properties can still change. Objects are compared by reference, not content.

YOUR TASK: Object practice

1. **Student management:** an object with name, marks object, and a method average().
2. **Product database:** an object of products keyed by id; add, update price, delete.
3. **Employee records:** an array of employee objects; list names, find by id, total salaries.
4. **Shopping cart data:** { items: [...], coupon: null }; compute total safely with optional chaining.
5. **Nested data manipulation:** update a deeply nested value immutably using spread.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `const a={x:1}; const b={x:1}; console.log(a===b);`

Q2. Output? `const a={n:{v:1}}; const b={...a}; b.n.v=9; console.log(a.n.v);`

Q3. What does `obj?.a?.b` return if obj.a is undefined?

Q4. Difference between Object.keys and for...in?

Q5. When do you need bracket notation?

Phase 10: Advanced Array + Object Problem Solving

10.1 A method for real data problems

Real data is usually an **array of objects**. Almost every task is a combination of a few moves:

Goal	Tool
Keep some records	filter
Change each record's shape	map
Find one record	find
Order records	sort (copy first)
One number from many (total, max)	reduce
Group or count	reduce into an object
Visit nested data	loops, recursion, optional chaining

Approach: (1) look at one record, (2) decide the output shape, (3) pick the tool, (4) write it step by step, (5) test with an empty array and a one-item array.

```
const orders = [
  { id: 1, customer: "Asha", total: 250, status: "paid" },
  { id: 2, customer: "Ravi", total: 90,  status: "pending" },
  { id: 3, customer: "Asha", total: 400, status: "paid" },
  { id: 4, customer: "Mina", total: 120, status: "paid" }
];

// filter + map
const paidIds = orders.filter(o => o.status === "paid").map(o => o.id); // [1,3,4]

// sort (copy first so original stays)
const byTotal = [...orders].sort((a, b) => b.total - a.total);

// total revenue
const revenue = orders.reduce((s, o) => s + o.total, 0);   // 860

// max
const biggest = orders.reduce((m, o) => (o.total > m.total ? o : m));

// group by customer
const byCustomer = orders.reduce((acc, o) => {
  (acc[o.customer] ||= []).push(o);
  return acc;
}, {});

// count frequencies
const counts = orders.reduce((acc, o) => {
  acc[o.status] = (acc[o.status] ?? 0) + 1;
  return acc;
}, {});   // { paid: 3, pending: 1 }
```

10.2 Nested data and API-like shapes

```
const api = {
  data: { users: [
    { id: 1, name: "Asha", skills: ["js", "css"], address: { city: "Delhi" } },
    { id: 2, name: "Ravi", skills: ["html"],       address: null }
  ] }
};

const cities = api.data.users.map(u => u.address?.city ?? "unknown");
const allSkills = api.data.users.flatMap(u => u.skills);   // flat list
const unique = [...new Set(allSkills)];                    // remove duplicates
const summary = api.data.users.map(({ id, name }) => ({ id, name }));
```

Common mistakes

- Mutating the original by calling sort or reverse directly.
- Forgetting the initial value in reduce (breaks on empty arrays).
- Forgetting to return the accumulator in reduce.
- Parentheses around an object literal in an arrow body: `x => ({ a: x })`.
- Not guarding against missing nested data.

YOUR TASK: Graduated problems (use the orders data above)

1. List the names of customers who have a paid order (no duplicates).
2. Compute the total per customer as an object.
3. Find the customer with the highest total spend.
4. Produce an array of strings like 'Asha: 650'.
5. Given students with an array of subject marks, return those whose average exceeds 60, sorted by average descending.
6. Given a nested menu (categories with items), return a flat list of items under 200 with their category name attached.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Why copy before sorting?

Q2. How do you remove duplicates from an array?

Q3. What shape does a group-by reduce produce?

Q4. What does flatMap do?

Q5. What happens with reduce on an empty array with no initial value?

Phase 11: The DOM

11.1 What the DOM is

When the browser loads HTML it builds the **DOM** (Document Object Model): a tree of objects representing every element, attribute and piece of text. HTML is the text file; the DOM is the live in-memory version that JavaScript can read and change. When you change the DOM, the screen updates. The entry point is the global `document` object.

```
<body>                document
  <h1>Hi</h1>   =>     +-- body
  <ul>                      +-- h1 -> "Hi"
    <li>A</li>              +-- ul
  </ul>                         +-- li -> "A"
</body>
```

Connection to what you know: CSS selectors you already use (`.card`, `#title`, `ul li`) are exactly what `querySelector` accepts. You are now selecting elements to *change* them instead of to *style* them.

11.2 Selecting elements

Method	Returns
document.getElementById('id')	One element or null
document.querySelector('css')	First match or null
document.querySelectorAll('css')	NodeList of all matches (empty if none). Supports forEach and for...of.
element.children, parentElement, nextElementSibling, previousElementSibling	Navigate the tree
element.closest('css')	Nearest ancestor (or itself) that matches

```
const title = document.querySelector("#title");
const items = document.querySelectorAll(".item");
items.forEach(el => console.log(el.textContent));
console.log(title.parentElement, title.nextElementSibling);
```

Common mistake: Selecting an element that does not exist yet (script runs before HTML) gives null, and `null.textContent` throws. Use defer or place scripts at the end of body.

11.3 Reading and changing content, attributes, classes, styles

Property	Behavior
textContent	Raw text of an element and descendants. Safe. Ignores CSS.
innerText	Text as visible on screen (respects CSS like display:none). Slower.
innerHTML	Parses a string as HTML. Powerful but dangerous with user input.

Security: If you put user-provided text into innerHTML, a malicious string such as an img tag with an onerror handler will execute (XSS attack). Use textContent for anything that is text. Use innerHTML only with markup you fully control.

```
const box = document.querySelector("#box");
box.textContent = "Hello";

// attributes
const link = document.querySelector("a");
link.setAttribute("href", "https://example.com");
console.log(link.getAttribute("href"));
link.removeAttribute("target");

// classList (best way to style dynamically)
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("dark");              // add if absent, remove if present
console.log(box.classList.contains("active"));

// inline styles (camelCase). Prefer classes for real styling.
box.style.backgroundColor = "gold";
```

11.4 Creating, inserting and removing elements

```
const list = document.querySelector("#list");
const li = document.createElement("li");   // not on page yet
li.textContent = "New item";
li.classList.add("item");
list.append(li);          // end of list (also accepts strings)
list.prepend("First");    // start of list
li.remove();              // delete it
list.replaceChildren();   // empty the list
oldNode.replaceWith(newNode);

// build many items from data
const names = ["Asha", "Ravi"];
names.forEach(n => {
  const el = document.createElement("li");
  el.textContent = n;
  list.append(el);
});
```

YOUR TASK: DOM practice

1. **Counter:** show a number in a span; update it from the console first.
2. **Dynamic cards:** from an array of products, create a card (div with h3 and p) for each and append them.
3. **Student list:** render an array of student objects as a list; highlight those with marks above 75 using classList.
4. **Product list:** render with price; show a different class for out-of-stock items.
5. **Search/filter UI (static for now):** given an array and a search string in a variable, render only matching items.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Difference between querySelector and querySelectorAll?

Q2. Why is textContent safer than innerHTML for user input?

Q3. What does classList.toggle('x') do?

Q4. Why can querySelector return null and what goes wrong?

Q5. What is the difference between HTML and the DOM?

Phase 12: Events

12.1 What events are and how to listen to them

An **event** is something that happens in the page: a click, a key press, typing, a form submit, a mouse move. The browser fires events constantly. To react, you attach an **event listener**: a function (callback) that the browser calls when that event happens on that element. This is the first time your code does not run top to bottom immediately; instead it runs *later*, when the user does something.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", function (event) {
  console.log("Clicked!", event.type);
});

// do not call the handler yourself:
btn.addEventListener("click", sayHi);     // correct: pass the function
// btn.addEventListener("click", sayHi()); // WRONG: calls it immediately
```

Why addEventListener beats inline onclick

- You can attach **multiple** listeners to the same event; onclick allows only one (a second replaces the first).
- Keeps JavaScript out of HTML (separation of concerns, easier to maintain).
- Gives control over options: capture phase, once, passive.
- Listeners can be removed with removeEventListener (needs the same function reference).
- Inline handlers run in a special scope and mix badly with modules.

Event	Fires when
click	Element is clicked
input	Value changes as the user types (every change)
change	Value is committed (blur for text, selection for select/checkbox)
submit	A form is submitted
keydown / keyup	A key is pressed / released
mouseover / mouseout / mousemove	Pointer enters / leaves / moves
DOMContentLoaded	HTML is fully parsed

12.2 The event object, target vs currentTarget, preventDefault

```
document.querySelector("#input").addEventListener("input", (e) => {
  console.log(e.target.value);       // what the user typed so far
});

document.addEventListener("keydown", (e) => {
  console.log(e.key);                // "Enter", "a", "Escape" ...
});

document.querySelector("form").addEventListener("submit", (e) => {
  e.preventDefault();                // stop the page reload
  console.log("handled in JS");
});
```

Property	Meaning
event.target	The element that actually triggered the event (deepest element clicked)
event.currentTarget	The element the listener is attached to
event.preventDefault()	Cancels the browser's default action (form submit, link navigation)
event.stopPropagation()	Stops the event from bubbling further up (use sparingly)

12.3 Bubbling, capturing and event delegation

When you click an element, the event travels in three phases: **capture** (window down to the target), **target**, then **bubble** (target back up to window). By default listeners run in the bubble phase, so a click on a button inside a div also triggers click listeners on the div and all ancestors. Pass `{ capture: true }` to listen during capture (rarely needed).

```
<ul id="list">
  <li>One</li><li>Two</li>
</ul>

// Delegation: ONE listener on the parent handles all children,
// including items added later.
document.querySelector("#list").addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li) return;                   // click was not on an li
  li.classList.toggle("done");
});
```

Why delegation? Fewer listeners (better memory), and it works for dynamically created elements; with direct listeners you would have to attach one to each new item.

Common mistakes

- Calling the function instead of passing it (`handler()`).
- Using arrow functions where you need this to be the element (arrow functions do not have their own this; prefer event.currentTarget).
- Forgetting preventDefault on form submit.
- Attaching listeners inside loops repeatedly or inside other listeners, creating duplicates.
- Removing a listener with a different function instance (anonymous functions cannot be removed).

YOUR TASK: Event practice

1. **Interactive buttons:** three buttons that change the background colour of a box.
2. **Counter:** plus, minus and reset buttons. Do not go below zero.
3. **Live character count:** show how many characters typed in a textarea (input event).
4. **Dynamic list:** input + Add button appends an li; pressing Enter also adds.
5. **Delegation:** click an li to mark it done; click a delete button inside it to remove it. Use one listener on the ul.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Difference between target and currentTarget?

Q2. What is bubbling?

Q3. Why use delegation?

Q4. What happens if you write `btn.addEventListener('click', go())`?

Q5. Difference between input and change events?

Phase 13: Forms & Validation

13.1 Forms, values and the submit event

A form groups inputs. When submitted, the browser reloads or navigates by default. In JavaScript apps we listen for the **submit** event on the **form** (not the button click, so Enter key and keyboard users work), call `preventDefault()`, read the values and validate.

```
<form id="signup" novalidate>
  <input id="email" type="email" required>
  <input id="age" type="number" min="1">
  <span id="error"></span>
  <button type="submit">Join</button>
</form>

const form = document.querySelector("#signup");
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const email = form.elements.email.value.trim();   // values are always STRINGS
  const age = Number(form.elements.age.value);
  console.log(email, age);
});
```

Input type	Read with	Notes
text, email, password, textarea, select	input.value	Always a string
number	Number(input.value)	Empty string becomes 0; check for '' first
checkbox	input.checked	Boolean
radio group	querySelector('input[name=x]:checked')?.value	Selected one

13.2 Built-in vs custom validation, showing errors

HTML gives free validation: `required`, `minlength`, `min/max`, `type='email'`, `pattern`. The browser blocks submission and shows a bubble. Use `novalidate` on the form when you want to take full control and show your own messages in JS. **Always validate in JavaScript too**, and remember: real security validation must also happen on the server because users can bypass front-end checks.

```
function validate(email, password) {
  const errors = {};
  if (!email) errors.email = "Email is required";
  else if (!email.includes("@")) errors.email = "Enter a valid email";
  if (password.length < 8) errors.password = "At least 8 characters";
  return errors;                       // empty object means valid
}

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const errors = validate(form.elements.email.value.trim(),
                          form.elements.password.value);
  document.querySelector("#error").textContent =
      Object.values(errors).join(", ");
  if (Object.keys(errors).length === 0) {
    const data = Object.fromEntries(new FormData(form));   // { email, ... }
    console.log("Valid", data);
  }
});
```

FormData basics: `new FormData(form)` collects all named fields. `Object.fromEntries(formData)` converts it into a plain object. Fields must have a `name` attribute to be included. Prefer showing errors next to the field and clearing old errors on each submit.

YOUR TASK: Form projects

1. **Registration form:** name, email, password, confirm password; validate every rule and show errors per field.
2. **Login form:** compare against a hard-coded user object; show success/failure message.
3. **Contact form:** name, email, message (min 20 chars) with a live character counter.
4. **Student registration system:** add students from a form into an array and render them in a table below; reject duplicates by email.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Why listen to submit on the form instead of click on the button?

Q2. What type is input.value for a number input?

Q3. Why is front-end validation not enough?

Q4. What do you need on inputs for FormData to include them?

Q5. What does preventDefault do on a submit event?

Phase 14: Asynchronous JavaScript

14.1 Synchronous vs asynchronous, blocking vs non-blocking

Synchronous code runs one step at a time; each step waits for the previous. Because JavaScript has a single call stack, a slow synchronous task **blocks** everything, including clicks and rendering.
Asynchronous code starts a slow task (timer, network request) and moves on; the result is handled later when ready. This keeps the page responsive.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
// Output: 1, 3, 2
// Even with 0 ms, the callback waits until the stack is empty.
```

14.2 The event loop (the model you must understand)

JavaScript runs on one thread. The browser provides extra machinery around it:

- **Call stack:** where your functions run, one at a time.
- **Web APIs:** browser features (timers, network, DOM events) that work outside the stack.
- **Callback (task) queue:** where finished timer/event callbacks wait.
- **Microtask queue:** where promise callbacks wait. It has higher priority than the task queue.
- **Event loop:** a constant check: if the call stack is empty, run all microtasks first, then take the next task from the task queue and push it onto the stack.

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);       // task queue
Promise.resolve().then(() => console.log("promise")); // microtask queue
console.log("end");
// Output: start, end, promise, timeout
```

Walkthrough: (1) 'start' runs on the stack. (2) setTimeout hands the timer to the browser; its callback will go to the task queue. (3) The promise callback goes to the microtask queue. (4) 'end' runs. (5) Stack is empty: the loop runs microtasks first ('promise'), then the next task ('timeout').

14.3 Callbacks, timers, promises

```
const id = setInterval(() => console.log("tick"), 1000);
setTimeout(() => clearInterval(id), 3500);       // stop after 3 ticks

// callback hell (nesting makes code hard to read)
getUser(1, (user) => {
  getOrders(user.id, (orders) => {
    getItems(orders[0], (items) => { /* ... */ });
  });
});
```

A **Promise** is an object representing a value that will arrive later. States: **pending**, then either **fulfilled** (success) or **rejected** (failure). It can only settle once.

```
function fakeApi(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => ok ? resolve({ name: "Asha" })
                        : reject(new Error("Server down")), 500);
  });
}

fakeApi(true)
  .then(user => { console.log(user.name); return user.name.length; })
  .then(len => console.log(len))        // each then gets the previous return
  .catch(err => console.error(err.message))  // catches any rejection above
  .finally(() => console.log("done"));        // runs either way
```

14.4 async/await

async/await is syntax on top of promises that makes asynchronous code read like synchronous code. An `async` function always returns a promise. `await` pauses *that function* (not the whole program) until the promise settles. Errors are handled with try/catch.

```
async function load() {
  try {
    const user = await fakeApi(true);       // waits here
    console.log(user.name);
  } catch (err) {
    console.error("Failed:", err.message);
  } finally {
    console.log("finished");
  }
}
load();

// sequential vs parallel
const a = await taskA();             // waits
const b = await taskB();             // then waits again
const [x, y] = await Promise.all([taskA(), taskB()]);  // runs together
```

Common mistakes

- Forgetting await, so you get a pending Promise instead of the value.
- Using await outside an async function (top-level await works only in modules).
- Forgetting to handle errors (unhandled rejection).
- Awaiting tasks one by one when they are independent (use Promise.all).
- Using setInterval without clearInterval.

YOUR TASK: Async practice

1. **Timer:** a countdown from 10 using setInterval that stops at 0.
2. **Fake API:** write fakeFetch(id) returning a promise that resolves after 1 second with a user object, or rejects for unknown ids.
3. **Sequential tasks:** call fakeFetch for ids 1, 2, 3 one after another using async/await.
4. **Parallel tasks:** do the same with Promise.all and compare the time taken.
5. **Error handling:** request an invalid id and show a friendly message with try/catch.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Predict the order: `console.log(1); setTimeout(()=>console.log(2),0); Promise.resolve().then(()=>console.log(3)); console.log(4);`

Q2. What are the three states of a Promise?

Q3. What does await actually pause?

Q4. What does an async function always return?

Q5. Which has priority, microtask or task queue?

Phase 15: Fetch & APIs

15.1 HTTP and JSON basics

The web works by **request and response**. A client (your browser) sends an HTTP request to a server; the server sends back a response. An **API** is a set of URLs (endpoints) that return data instead of web pages, usually as **JSON**.

Part	Meaning
Method	GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
URL	Where to send the request
Headers	Metadata, e.g. Content-Type: application/json, Authorization
Body	Data sent (POST/PUT)
Status code	200 OK, 201 Created, 400 bad request, 401 unauthorized, 404 not found, 500 server error

JSON is a text format that looks like JavaScript objects but is stricter: keys must be in double quotes, no functions, no comments. `JSON.stringify(obj)` converts an object to a JSON string; `JSON.parse(text)` converts back.

15.2 Using fetch

`fetch(url)` returns a **Promise** of a Response. Reading the body (response.json()) is *also* a promise, so you await twice.

```
async function getUsers() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) {                                  // 404, 500 do NOT throw!
      throw new Error(`HTTP ${res.status}`);
    }
    const users = await res.json();
    return users;
  } catch (err) {
    console.error("Request failed:", err.message);
    return [];
  }
}
```

Key gotcha: fetch only rejects on network failure. A 404 or 500 response still resolves, so you must check `res.ok` yourself.

```
// POST with JSON body
async function createPost(title) {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body: "hello", userId: 1 })
  });
  if (!res.ok) throw new Error("Failed");
  return res.json();
}
```

Showing data and UX states

Real UIs handle three states: **loading** (show a spinner or text), **success** (render data) and **error** (show a message and let the user retry). Always render API text with textContent unless you trust the data.

```
async function showUsers() {
  const out = document.querySelector("#out");
  out.textContent = "Loading...";
  const users = await getUsers();
  out.replaceChildren();
  users.forEach(u => {
    const li = document.createElement("li");
    li.textContent = `${u.name} (${u.email})`;
    out.append(li);
  });
}
```

YOUR TASK: API practice

1. **Fetch users:** list names and emails from JSONPlaceholder with loading and error states.
2. **Search users:** an input that filters the fetched list as you type.
3. **Product API:** fetch products from a public API (e.g. dummyjson.com) and render cards.
4. **Weather-style project:** input a city, fetch from a free weather API, show temperature and a description; handle 'city not found'.
5. **CRUD practice:** use JSONPlaceholder to GET posts, POST a new one, PUT/PATCH an edit and DELETE one (responses are simulated).

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Does fetch reject on a 404?

Q2. Why do we await twice with fetch?

Q3. Which header tells the server you are sending JSON?

Q4. Difference between JSON.parse and JSON.stringify?

Q5. Which HTTP method is for creating a resource?

Phase 16: Modern JavaScript

16.1 Destructuring, spread, rest, ?. and ??

You already met these in passing. Now use them fluently.

```
// destructuring
const [a, , c = 0] = [1, 2];                // skip, default
const { name, role = "user", ...others } = { name: "A", x: 1, y: 2 };
function show({ title, price = 0 }) { return `${title}: ${price}`; }

// spread: copy and merge (shallow)
const merged = { ...defaults, ...userSettings };   // later wins
const copy = [...list, newItem];

// rest: gather
const [head, ...tail] = [1, 2, 3];
const sum = (...n) => n.reduce((s, x) => s + x, 0);

// ?. and ??
const city = user?.address?.city ?? "Unknown";
count ??= 0;          // assign only if null/undefined
```

16.2 Modules: import and export

Modules let you split code into files with their own scope. Nothing leaks to global unless exported. Load a module with `<script type='module' src='app.js'>` (modules are deferred automatically and need a server, not just opening the file).

```
// math.js
export const PI = 3.14;               // named export
export function add(a, b) { return a + b; }
export default function multiply(a, b) { return a * b; }   // one default

// app.js
import multiply, { add, PI } from "./math.js";
import * as math from "./math.js";     // namespace import
import { add as sum } from "./math.js"; // rename
```

Named exports: many per file, imported with braces and exact names. **Default export**: one per file, imported without braces under any name. Prefer named exports for clarity.

16.3 Set, Map and Date

Tool	Purpose	Key methods
Set	Collection of unique values	add, has, delete, size, iterable
Map	Key-value pairs where keys can be ANY type; keeps insertion order	set, get, has, delete, size, iterable
Date	Dates and times	new Date(), getFullYear, getMonth (0-based!), getDate, getTime, toISOString, toLocaleDateString

```
const unique = new Set([1, 2, 2, 3]);       // {1, 2, 3}
unique.add(4); console.log(unique.has(2), unique.size);

const m = new Map();
m.set("a", 1).set(2, "two");
for (const [k, v] of m) console.log(k, v);

const d = new Date("2026-10-14");
console.log(d.getFullYear(), d.getMonth() + 1);   // month is 0-11
console.log(d.toLocaleDateString("en-IN"));
```

Map vs object: use an object for fixed records; use a Map for dynamic lookups, non-string keys, or when you frequently add/remove keys. **Other useful syntax:** `Array.from`, `Object.fromEntries`, `array.at(-1)`, `structuredClone`, `Array.prototype.flatMap`, numeric separators `1_000_000`, `String.prototype.padStart`.

YOUR TASK: Modern JS practice

1. Refactor an old function using destructuring parameters and defaults.
2. Deduplicate and sort a list of tags using Set and spread.
3. Build a word frequency Map from a sentence and print the top 3.
4. Split a small app into two module files (utils.js and main.js) and run it with type='module'.
5. Print how many days remain until a given date using Date.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Output? `const {a, ...r} = {a:1,b:2,c:3}; console.log(r);`
Q2. Why is new Date(2026, 0, 1) January 1st?
Q3. Named vs default export?
Q4. What does `[...new Set([1,1,2])]` give?
Q5. When prefer Map over a plain object?

Phase 17: Error Handling & Debugging

17.1 Kinds of errors

Kind	When	Example	How you notice
Syntax error	Code cannot be parsed	Missing bracket or quote	Nothing runs; red error with line number
Runtime error	Valid code fails while running	Calling undefined, reading a property of null	Red error in the console at that moment
Logical error	Runs fine, wrong result	< instead of <=, wrong formula	No error. Wrong output. Hardest to find.

Common built-in error types: **SyntaxError**, **ReferenceError** (name does not exist), **TypeError** (wrong type/operation, e.g. cannot read properties of undefined), **RangeError** (value out of range / too much recursion).

17.2 try / catch / finally and throw

```
function parseAge(text) {
  const n = Number(text);
  if (Number.isNaN(n)) throw new Error("Age must be a number");
  if (n < 0) throw new RangeError("Age cannot be negative");
  return n;
}

try {
  console.log(parseAge("abc"));
} catch (err) {
  console.error(err.name, err.message);   // Error  Age must be a number
} finally {
  console.log("always runs");
}
```

Use try/catch around code that can fail for reasons outside your control (network, JSON.parse, user input). Do **not** use it to hide bugs, and do not wrap everything. throw new Error(...) lets *you* signal a problem; catch decides how to recover or show a message. Note try/catch does not catch errors inside callbacks that run later, unless you handle them inside (or use await in an async function).

17.3 Debugging with DevTools

- **Read the error:** type, message, file:line. Click the link to jump to the exact line.
- **console.log / console.table / console.error:** quick checks. Label values: `console.log('total', total)`.
- **Breakpoints:** Sources tab, click a line number. Execution pauses there; inspect variables in Scope, hover values.
- **Step controls:** Step over (next line), Step into (enter a function), Step out, Resume.
- **debugger;** statement acts as a breakpoint in code.
- **Watch and Call Stack panels:** track expressions and see how you got here.
- **Network tab:** see requests, status codes and responses for fetch.
- **Elements tab:** inspect the live DOM and applied CSS.

Debugging strategy

1. Reproduce the bug reliably.
2. Read the error message fully before changing anything.
3. Narrow it down: where do the values first become wrong? Bisect by logging or using breakpoints.
4. Form a hypothesis, change **one** thing, test.
5. Fix the cause, not the symptom. Add a check so it does not come back.
6. Explain the bug aloud (rubber duck debugging); it often reveals it.

YOUR TASK: Debug these (find and explain each bug)

1. **Bug 1:** `const nums=[1,2,3]; let total; nums.forEach(n => total += n); console.log(total);` prints NaN. Why?
2. **Bug 2:** `for (var i=0;i<3;i++){ setTimeout(()=>console.log(i),0); }` prints 3 3 3. Why, and how do you fix it?
3. **Bug 3:** `const btn=document.querySelector('#save'); btn.addEventListener('click', save());` runs save immediately. Why?
4. **Bug 4:** `const users=[{n:'A'}]; const u=users.find(x=>x.n==='B'); console.log(u.n);` throws. Why, and how do you guard it?
5. **Bug 5:** `function avg(a){ return a.reduce((s,x)=>s+x)/a.length }; avg([])` fails. Why?

MINI QUIZ (answers are in the Answer Key at the end)

Q1. Which error type: using a variable that was never declared?

Q2. Which error type: `undefined.name`?

Q3. Does finally run if try succeeds?

Q4. What is a logical error and why is it hard?

Q5. What does a breakpoint do?

Phase 18: JavaScript Projects

For every project follow the same loop: **1. Requirements** (what must it do?), **2. Features** (break into small pieces), **3. Data structure** (what does the state look like?), **4. Build step by step** (one feature, test, next), **5. Review**, **6. Refactor** (rename, remove duplication, extract functions). Write most of the code yourself; use hints before solutions.

Core pattern for UI projects

```
// 1. State (single source of truth)
let state = { items: [] };

// 2. Render (draw the UI from the state)
function render() { /* clear list, create elements from state.items */ }

// 3. Update (change state, then re-render)
function addItem(text) {
  state = { ...state, items: [...state.items, { id: Date.now(), text, done: false }] };
  render();
}

// 4. Events call update functions
form.addEventListener("submit", (e) => { e.preventDefault(); addItem(input.value.trim()); });
```

Project list with feature breakdown

#	Project	Key features	Data structure / concepts
1	Counter	Increase, decrease, reset; no negatives; colour by sign	number; click events; classList
2	Calculator	Digits, operators, clear, decimal, equals; avoid divide by zero	strings for display; event delegation; functions
3	Digital clock	Live time, 12/24h toggle, date	Date; setInterval; padStart
4	Character counter	Live chars/words, limit warning	input event; string methods
5	Colour changer	Random colour, copy hex, history	Math.random; arrays; clipboard
6	To-do app	Add, complete, delete, filter, edit, count left	array of {id,text,done}; delegation; render()
7	Quiz app	Questions, score, next, results, restart	array of question objects; index state
8	Expense tracker	Add/delete expenses, categories, totals, filters	array of objects; reduce; forms
9	Student management	CRUD students, search, sort, average marks	array of objects; sort/filter; validation
10	Product filter/search	Search text, category, price range, sort	filter chain; derived state
11	Weather app	City search, fetch API, loading/error states	fetch; async/await; JSON
12	Shopping cart	Product list, add/remove, quantity, totals, coupon	state; reduce; nested objects
13	Notes app	Create, edit, delete, search, persistence	localStorage + JSON; delegation
14	CRUD app	Full create/read/update/delete with an API	fetch; POST/PUT/DELETE; async state
15	Dashboard	Several widgets (stats, list, chart-like bars) from API data	modules; fetch; data transformation

Persistence tip: localStorage stores only strings: use `localStorage.setItem('k', JSON.stringify(data))` and `JSON.parse(localStorage.getItem('k')) ?? []`.

Worked planning example: To-do app

1. **Requirements:** add a task, mark done, delete, filter All/Active/Done, show remaining count, keep tasks after refresh.
2. **Features:** (a) form to add, (b) render list, (c) toggle done, (d) delete, (e) filter buttons, (f) counter, (g) save/load.
3. **Data:** `tasks = [{ id, text, done }]` and `filter = 'all' | 'active' | 'done'`.
4. **Build order:** data + render, add, toggle/delete with delegation, filter, counter, persistence.
5. **Refactor:** split into functions (addTask, toggleTask, deleteTask, getVisibleTasks, save, load); remove duplicated code; rename unclear variables.

YOUR TASK: Start here

1. Plan the Counter project in writing: requirements, features, state.
2. Build it, then send me the code for review.
3. After my review, refactor and move to the Calculator.

Phase 19: Interview & Coding Preparation

19.1 A 7-step method for any problem

1. **Understand:** restate the problem in your own words.
2. **Input / output:** types, examples, constraints.
3. **Edge cases:** empty input, one item, duplicates, negative numbers, wrong types, very large input.
4. **Approach:** describe the steps in plain language (pseudocode) before code.
5. **Write code:** small named functions, clear names.
6. **Test manually:** trace with a normal case and each edge case.
7. **Optimize:** only if needed; mention time/space complexity.

19.2 Practice problems (try each without looking for answers)

Strings

- Reverse a string. Check palindrome ignoring case and punctuation.
- Count vowels. Find the most frequent character.
- Capitalize the first letter of each word.
- Are two strings anagrams?
- Compress 'aaabbc' to 'a3b2c1'.

Arrays and loops

- Find the second largest number.
- Remove duplicates without using Set, then with Set.
- Two Sum: indices of two numbers adding to a target.
- Rotate an array by k.
- Flatten a nested array one level, then fully (recursion).
- Chunk an array into groups of n.

Objects and data

- Group an array of people by age.
- Count word frequency and return the top 3.
- Merge two objects deeply (nested).
- Given orders, return total spent per customer.
- Invert an object (swap keys and values).

Functions

- Implement your own map, filter and reduce.
- Write once(fn) so fn runs only the first time.
- Write a debounce function (conceptually: wait for the user to stop typing).
- Compose and pipe functions.

DOM, events and forms

- Build a to-do list with delegation.

- Live search/filter over a list.
- Accordion (one open at a time), tabs, modal open/close with Escape.
- Form validation with per-field errors.
- Star rating component.

Async and debugging

- Fetch users and render them with loading and error states.
- Run three promises in parallel and sequentially; compare.
- Predict outputs mixing setTimeout, promises and sync code.
- Given buggy code, find and fix three bugs.

19.3 How to approach one problem (example)

Problem: 'Return the first non-repeating character in a string; return null if none.'

1. Understand: for 'swiss' the answer is 'w'. For 'aabb' it is null.
2. I/O: string in, character or null out.
3. Edge cases: empty string, all repeated, one character, case sensitivity (ask or state an assumption).
4. Approach: count each character in an object; then loop again in original order and return the first with count 1.
5. Code it. Test with 'swiss', 'aabb', '', 'a'.
6. Complexity: two passes: O(n) time, O(k) space for k distinct characters.

YOUR TASK: Your first interview problem

1. Solve 'first non-repeating character' yourself using the steps above, then send the code.
2. Tell me three edge cases you tested and what you expected for each.

MINI QUIZ (answers are in the Answer Key at the end)

Q1. What is the time complexity of searching an unsorted array with a loop?
Q2. Why use an object/Map for counting instead of nested loops?
Q3. Name three edge cases to test for almost any function.
Q4. What does debounce do?
Q5. Why explain your approach before coding in an interview?

Phase 20: Final Revision

20.1 JavaScript cheat sheet

Topic	Remember
Variables	const by default, let when reassigning, avoid var
Types	string, number, boolean, undefined, null, bigint, symbol; typeof null is 'object'
Equality	Use === and !==; == coerces types
Falsy	false, 0, -0, 0n, '', null, undefined, NaN (everything else truthy)
Logical	&& first falsy; || first truthy; ?? only null/undefined
Scope	let/const block scoped; var function scoped; lexical scope chain
Hoisting	Function declarations fully; var as undefined; let/const in TDZ
Functions	Values you can pass/return; arrow functions have no own this; prefer pure logic
Copying	Primitives by value; objects/arrays by reference; spread is shallow; structuredClone is deep
Async	Microtasks (promises) before tasks (timers); await only pauses its function
fetch	Check res.ok; await res.json(); handle errors with try/catch

20.2 Important syntax sheet

```
const add = (a, b = 0) => a + b;
for (let i = 0; i < n; i++) {}      for (const x of arr) {}      for (const k in obj) {}
const { a, b: renamed, ...rest } = obj;      const [x, , z = 1] = arr;
const copy = { ...obj, extra: 1 };           const list = [...arr, item];
value ?? fallback;     obj?.a?.b;     cond ? yes : no;
switch (v) { case 1: ...; break; default: ... }
try { ... } catch (e) { ... } finally { ... }
async function f() { const r = await fetch(url); return r.json(); }
el.addEventListener("click", (e) => { e.preventDefault(); });
import { a } from "./m.js";     export const b = 1;     export default fn;
```

20.3 Important methods sheet

Group	Methods (mutating ones in bold)
String	length, at, slice, substring, indexOf, includes, startsWith, endsWith, trim, toUpperCase, toLowerCase, replace, replaceAll, split, padStart, repeat
Array (mutating)	**push, pop, shift, unshift, splice, sort, reverse, fill**
Array (non-mutating)	slice, concat, map, filter, reduce, find, findIndex, some, every, includes, indexOf, flat, flatMap, join, at, forEach
Object	Object.keys, values, entries, assign (mutates target), fromEntries, freeze; spread; structuredClone
Number / Math	Number(), parseInt, parseFloat, toFixed, Number.isNaN, Math.round/floor/ceil/max/min/random/abs
DOM	querySelector(All), getElementById, createElement, append, prepend, remove, replaceWith, textContent, classList, setAttribute, closest
Timers / JSON	setTimeout, setInterval, clearTimeout, clearInterval, JSON.parse, JSON.stringify

20.4 Common interview questions

1. What is the difference between var, let and const?
2. Explain hoisting and the temporal dead zone.
3. What is a closure? Give a practical use.
4. == vs ===? Examples of coercion surprises.
5. Explain scope, scope chain and lexical scope.
6. How does the event loop work? Order of setTimeout vs Promise.
7. map vs forEach; find vs filter; slice vs splice.
8. What is event bubbling and delegation? Why use it?
9. textContent vs innerHTML. What is XSS?
10. What are promises? What does async/await do under the hood?
11. What is a pure function? Why does it matter?
12. Shallow vs deep copy.
13. How does fetch handle HTTP errors?
14. What is this and how do arrow functions treat it?
15. Explain debounce and throttle.

20.5 Output prediction questions

MINI QUIZ (answers are in the Answer Key at the end)

Q1. `console.log(typeof NaN, typeof [], typeof null)`

Q2. `console.log([1,2,3] + [4])`

Q3. `console.log(0.1 + 0.2 === 0.3)`

Q4. `let a = 1; { let a = 2; } console.log(a);`

Q5. `console.log([] + {}, [] == ![])`

Q6. `var a = 1; function f(){ console.log(a); var a = 2; } f();`

Q7. `for (var i=0;i<3;i++) setTimeout(()=>console.log(i));`

Q8. `for (let i=0;i<3;i++) setTimeout(()=>console.log(i));`

Q9. `console.log([1,2,3].map(parseInt))`

Q10. `console.log('b' + 'a' + +'a' + 'a')`

20.6 Common mistakes checklist

- Mutating the original array/object by accident (sort, reverse, splice, assignment of references).
- Forgetting return in arrow functions with braces or in reduce.
- Using = instead of === in conditions.
- Selecting DOM elements before they exist; not checking for null.
- Forgetting preventDefault on submit.
- Using innerHTML with user data.
- Not awaiting a promise; not checking res.ok.
- Default sort() on numbers.
- Off-by-one errors in loops and slice.
- Treating input.value as a number.

20.7 Mini coding tests, DOM and event challenges, mixed tests

Mini coding tests (30 minutes each)

- **Test A (strings and arrays):** word frequency; top 3 words; capitalise; palindromes in a sentence.
- **Test B (objects):** from an array of orders compute revenue per customer, best customer, average order value, and a sorted summary.
- **Test C (functions):** implement map, filter, reduce, once and debounce yourself.

DOM challenges

- Render a product grid from an array; add a 'sold out' badge.
- Tabs component with keyboard support.
- Accordion where only one panel is open.
- Modal that closes on Escape and on outside click.
- Table sortable by clicking headers.

Event challenges

- Event delegation: a list where each item has Done and Delete buttons.
- Throttle a scroll listener.
- Drag items to reorder (mouse events).
- Keyboard shortcuts (Ctrl+K opens search).

Mixed JavaScript tests

- Build a searchable user directory from an API with loading/error states and a favourites list stored in localStorage.
- Build a quiz app with timer, score and a results screen.
- Build a shopping cart with quantities, coupon codes, totals and persistence.

Where to go next: once the fundamentals and the projects are solid, learn tooling (npm, bundlers, Git) and then a framework such as React. Frameworks are much easier once DOM, events, state and async are second nature.

Answer Key (Mini Quizzes)

Try each question on paper first, then check here.

Phase 0: JavaScript & Browser Fundamentals

Q1. JavaScript.

Q2. External: separation of concerns, reuse, caching, easier maintenance.

Q3. Downloads the script while HTML is parsed but runs it only after parsing finishes, so elements exist.

Q4. Top to bottom, one statement at a time.

Q5. It turns silent mistakes (like accidental globals) into visible errors.

Phase 1: Variables, Values & Data Types

Q1. 5 (primitives copy by value).

Q2. var is hoisted and initialized as undefined; let is hoisted but stays in the temporal dead zone until its line.

Q3. 'object'. It is a long-standing bug kept for backward compatibility.

Q4. undefined: no value assigned yet (set by JS). null: intentionally empty (set by the programmer).

Q5. Yes. const prevents reassigning arr, not changing its contents.

Phase 2: Operators & Expressions

Q1. '52' and 3

Q2. 'a' and 0

Q3. true and 't' ([] is truthy but == coerces both sides to numbers: 0 == 0).

Q4. 2 + 4 = 6

Q5. NaN is defined as not equal to anything, even itself. Use Number.isNaN(value).

Phase 3: Conditional Logic

Q1. A. The string '0' is truthy.

Q2. Execution falls through into the following cases until a break or the end.

Q3. It assigns 5 to x and the condition is always truthy. Use ===.

Q4. Return early on invalid cases; main logic stays flat and readable.

Q5. Many exact matches of one value, no ranges.

Phase 4: Loops & Iteration

Q1. 0, 2, 4

Q2. do...while

Q3. break exits the loop; continue skips to the next iteration.

Q4. 12

Q5. for...in for object keys; for...of for values of arrays/strings.

Phase 5: Functions

Q1. undefined

Q2. Parameter: name in the definition. Argument: actual value passed in the call.

Q3. No. Output depends on randomness, so same input can give different outputs.

Q4. A function passed as an argument to be called later by another function.

Q5. 8 (implicit return when there are no braces).

Phase 6: Scope, Hoisting & Execution Concepts

Q1. 'number' 'undefined' (b is out of scope; typeof an undeclared name is 'undefined').

Q2. 'ok'

Q3. undefined (local var v is hoisted inside t and shadows the outer v).

Q4. Name visibility is decided by where code is written in the source, not by where a function is called.

Q5. Too many nested calls, usually recursion without a stopping condition.

Phase 7: Strings

Q1. 'llo'

Q2. 'abc' (immutable; result was discarded).

Q3. 4 (double space gives an empty string element).

Q4. indexOf returns 0 (falsy) when found at the start, and -1 (truthy) when not found.

Q5. 'a-b-c'.replaceAll('-', ' ') or split('-').join(' ')

Phase 8: Arrays

Q1. [1,2,3] [2,4,6]

Q2. undefined

Q3. [1,4,5]

Q4. true

Q5. [1,2,3] (same reference)

Q6. arr.sort((a, b) => a - b)

Phase 9: Objects

Q1. false (different references)

Q2. 9 (shallow copy shares nested objects)

Q3. undefined, without throwing.

Q4. keys returns an array of own enumerable keys; for...in also visits inherited enumerable keys.

Q5. Key in a variable, computed, or not a valid identifier (spaces, hyphens, digits first).

Phase 10: Advanced Array + Object Problem Solving

Q1. sort mutates the original array.

Q2. [...new Set(arr)]

Q3. An object whose keys are group names and values are arrays (or counts).

Q4. map followed by flat of depth 1.

Q5. TypeError.

Phase 11: The DOM

Q1. First match or null versus a NodeList of all matches.

Q2. textContent treats everything as plain text, so injected HTML/scripts cannot run.

Q3. Adds x if missing, removes it if present.

Q4. No match (or the element is not parsed yet); using the result then throws a TypeError.

Q5. HTML is the source text; the DOM is the live object tree the browser builds and JS manipulates.

Phase 12: Events

Q1. target: element that triggered it; currentTarget: element the listener is attached to.

Q2. An event propagating from the target up through its ancestors.

Q3. One listener for many children and it works for elements added later.

Q4. go runs immediately and its return value (probably undefined) is registered as the listener.

Q5. input fires on every edit; change fires when the value is committed.

Phase 13: Forms & Validation

Q1. Submit also fires for Enter key and assistive tech; click would miss those.

Q2. String.

Q3. Users can bypass it; the server must validate too.

Q4. A name attribute.

Q5. Stops the page reload/navigation so JS can handle the data.

Phase 14: Asynchronous JavaScript

Q1. 1, 4, 3, 2

Q2. pending, fulfilled, rejected.

Q3. Only the enclosing async function; the rest of the program keeps running.

Q4. A Promise.

Q5. Microtasks run first (all of them) before the next task.

Phase 15: Fetch & APIs

Q1. No, only on network failure. Check response.ok.

Q2. fetch returns a promise for the response; response.json() returns another promise for the parsed body.

Q3. Content-Type: application/json

Q4. parse: string to object. stringify: object to string.

Q5. POST

Phase 16: Modern JavaScript

Q1. { b: 2, c: 3 }

Q2. Month is zero-based: 0 = January.

Q3. Named: many per file, imported with exact names in braces. Default: one per file, any import name.

Q4. [1, 2]

Q5. Dynamic keys, non-string keys, frequent add/remove, guaranteed insertion order.

Phase 17: Error Handling & Debugging

Q1. ReferenceError

Q2. TypeError

Q3. Yes, finally always runs.

Q4. Code runs but gives wrong results; there is no error message to point at it.

Q5. Pauses execution at that line so you can inspect variables and step through.

Phase 19: Interview & Coding Preparation

Q1. O(n)

Q2. Constant-time lookups give O(n) instead of O(n squared).

Q3. Empty input, single item, duplicates/invalid values.

Q4. Delays running a function until the user stops triggering it for a set time.

Q5. Shows structured thinking, catches misunderstandings early, and lets the interviewer guide you.

Phase 20: Final Revision

Q1. number object object

Q2. '1,2,34' (both converted to strings)

Q3. false

Q4. 1

Q5. '[object Object]' and true

Q6. undefined

Q7. 3 3 3

Q8. 0 1 2 (let creates a new binding each iteration)

Q9. [1, NaN, NaN] (map passes index as the second argument, the radix)

Q10. 'baNaNa'